VELLORE INSTITUTE OF TECHNOLOGY

SCHOOL OF COMPUTER SCIENCE ENGINEERING AND INFORMATION SYSTEMS



A Case Study

on

Rule-Based Resource Placement and Service-Chaining Algorithm for MSP Deployment Requests

(Multi-Service Provider approach for RAN, E2E, and IMS Core deployment)

Submitted By

Vansh

23BIT0058

SCORE

Under the Guidance of

Prof. Suresh V J

SCORE

VIT

1. ABSTRACT

This extended case study develops the Rule-Based Multi-AZ Resource Placement and Service-Chaining algorithm (R-MRP) for the 5G Multi-Service Provider (MSP) strategy on AWS. A tenant request is converted into a deployment plan through mandatory rules for quota, resource capacity, tenant compatibility, failure-domain separation, ordered connectivity, link bandwidth and latency. Preference scores cannot override a mandatory rule. The planner reserves resources on a private snapshot, explores alternatives through bounded backtracking, and returns a complete plan only after all replica reservations succeed. A reproducible Python prototype evaluates synthetic infrastructure and service requests; its results establish the behavior of the planning logic rather than the performance of a deployed 5G network. The proposed architecture adds identity verification, AWS provisioning, runtime isolation and service validation around that planning core. It distinguishes RAN-to-UPF user traffic, 5G control-plane dependencies and IMS signaling. The principal contribution is an explainable admission and placement procedure with traceable decisions, a numerical worked example, failure handling and a clear evaluation framework.

2. INTRODUCTION

A 5G MSP platform allows several logical service providers to use a common infrastructure foundation while retaining separate subscribers, service policies, identities and traffic boundaries. In this report, MSP retains the meaning Multi-Service Provider used in the original case study. It does not imply that every network function or data path is shared between tenants. The cloud platform supplies infrastructure; each tenant receives only the resources and connectivity authorized by its service descriptor.

The original architecture covered RAN, end-to-end 5G connectivity and IP Multimedia Subsystem (IMS) services. Its third objective was to develop a rule-based placement and service-chaining algorithm. This extension develops that objective into a concrete engineering design: input and output models, mandatory constraints, deterministic scoring, placement search, path reservation, recovery behavior and executed local tests. The overall AWS deployment remains a proposed architecture.

The central difficulty is that compute placement and connectivity are coupled. Two nodes may have enough CPU and memory but lack a permitted route, adequate bandwidth or acceptable delay. A high-scoring node in a single Availability Zone (AZ) may also violate the requested replica separation. Therefore, the planner must assess the complete tenant service graph and resource state before declaring a request admissible.

3GPP defines distinct control and user-plane interfaces. The user path is RAN to UPF over N3 and UPF to a data network over N6; AMF and SMF are control functions [1]. Service chaining describes an ordered sequence of service functions and the forwarding path that realizes that order [2]. These distinctions prevent an attractive architecture diagram from hiding an incorrect packet path.

The proposed system is intended for an academic demonstrator with simulated RAN traffic, not physical radio access. Its design can guide a later AWS implementation, but radio delay, packet throughput, IMS call quality, cloud cost and real recovery time require separate measurement. Local algorithm outputs and assumed planning inputs are identified explicitly throughout the report.

3. PROBLEM STATEMENT

Given an authenticated MSP request and a versioned infrastructure inventory, determine whether the requested functions and their required connections can be placed without violating mandatory policy or exhausting available resources. If feasible within the configured search budget, return a deterministic plan with node assignments, replica zones, ordered paths, resource reservations, policy intent and validation evidence. Otherwise return a reasoned rejection or a search-budget outcome without changing the live inventory.

Item	Extended definition
Input	Tenant identity; request ID; function graph; per-function CPU, RAM and disk; traffic demand; quota; allowed zones; replica count; path delay bound; isolation and service policy.
Infrastructure state	Host roles, residual capacities, tenant compatibility, AZ labels, directed links, residual bandwidth, assumed delay and inventory version.
Output	Admitted plan or explicit rejection; per-replica placement; path list; reservations; decision trace; required deployment and monitoring actions.
Hard constraints	Authorized tenant, quota, supported roles, sufficient cumulative resources, permitted paths, bandwidth, delay and replica separation.
Soft preferences	Residual headroom, estimated locality and cost proxy after mandatory rules are satisfied.
Scope of local evidence	Synthetic planning and validation. No AWS provisioning, 5G packet forwarding or IMS registration is claimed.

A valid plan must hold for the entire request. Checking each function against the original free capacity independently can overbook a host. Similarly, verifying each chain against the original link bandwidth can admit multiple chains whose combined demand exceeds the link. R-MRP uses tentative residual state so later decisions see earlier reservations.

4. MOTIVATION OF THE CASE STUDY

Manual telecom deployment involves compute configuration, network policies, subscriber or service dependencies, monitoring and recovery procedures. Repeating these steps for each provider creates opportunities for inconsistent resource accounting and accidental cross-tenant access. A descriptor-driven planner makes the admission decision reviewable before deployment.

• **Operational motivation:** produce the same normalized plan for the same request, policy configuration and inventory state, with stable tie-breaking and a recorded explanation.

• **Technical motivation:** combine compute feasibility with ordered paths, bandwidth accounting and fault-domain rules instead of treating scheduling as a CPU-only problem.

• **Academic motivation:** connect graph modeling, constraints, greedy ranking, bounded search, software testing, cloud networking and telecom architecture in one reproducible case study.

AWS documents cloud deployment patterns for 5G network functions and distinguishes resilient regional control functions from latency-sensitive user-plane placement closer to users [3]. These patterns motivate the architecture without proving that any particular synthetic placement achieves production telecom performance.

5. OBJECTIVES

1. Design an AWS-native MSP architecture for simulated RAN, 5G core connectivity and IMS service dependencies.

2. Define explicit identity, network, workload and resource boundaries for each provider.

3. Develop an explainable rule-based resource-placement and service-chaining algorithm for MSP deployment requests.

4. Enforce cumulative capacity, ordered connectivity, latency, bandwidth and mandatory multi-AZ replica rules.

5. Specify deterministic ranking, bounded backtracking, atomic reservations and actionable rejection reasons.

6. Implement and execute a local reference planner using synthetic fixtures and meaningful acceptance and rejection tests.

7. Separate demonstrated algorithm behavior from proposed deployment targets and identify the work needed for AWS and telecom validation.

6. METHODOLOGY

The study follows a design-and-validation method. First, official telecom and cloud documentation defines the architectural boundaries. Second, MSP requirements and infrastructure are expressed as structured data. Third, mandatory constraints and preference rules are specified independently. Fourth, a local planner implements the admission and reservation logic. Finally, tests exercise valid requests and deliberately invalid conditions, and a worked trace explains the resulting placement.

Stage	Method	Evidence produced
Architecture	Separate user-plane, control-plane and IMS dependencies.	Layered architecture and interface diagram.
Modeling	Represent functions, hosts, zones and links explicitly.	Request schema and synthetic resource inventory.
Algorithm design	Filter hard constraints, rank, search and reserve.	Rules, pseudocode and complexity analysis.
Prototype	Run Python against deterministic fixtures.	Executable planner and machine-readable results.
Evaluation	Check expected admission, rejection and state invariants.	Test matrix, numerical trace and limitations.

The reference planner deliberately models a smaller problem than a complete telecom orchestrator. It places an UPF role and a DATA role in each complete replica chain, reached from a fixed INGRESS endpoint. It uses tenant-labeled hosts, four host resource dimensions and fixed shortest-latency routes. It does not emulate 3GPP protocols. The proposed implementation architecture adds authentication, descriptor translation, AWS provisioning, explicit traffic steering and operational health checks around that planning core.

6.1 Proposed Framework / Architecture

The framework has five responsibilities: provider request intake, a policy and placement layer, tenant workloads, a forwarding layer and infrastructure operations. Placement produces intent. Provisioning creates resources. Forwarding establishes the actual packet path. Monitoring then verifies the operational service, because a valid static plan does not establish runtime correctness.

6.1 Proposed Framework / Architecture

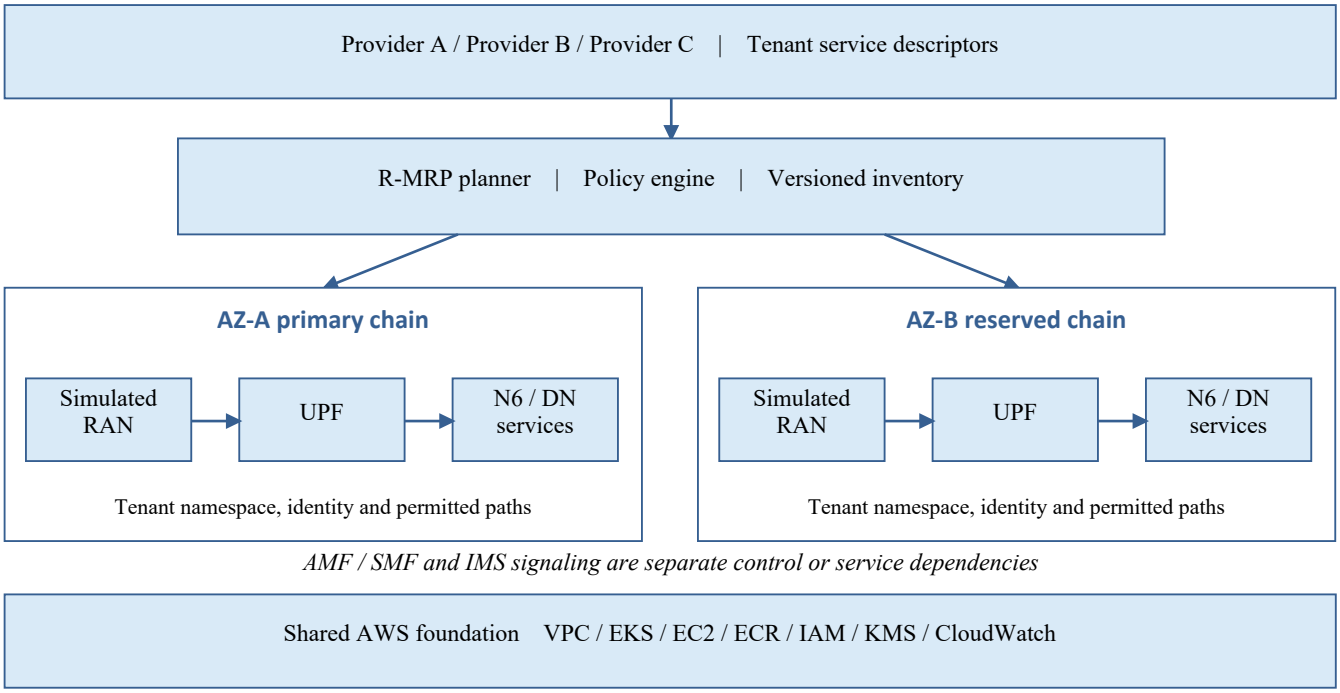


Fig. 1. Extended MSP architecture with explicit placement and tenant boundaries.

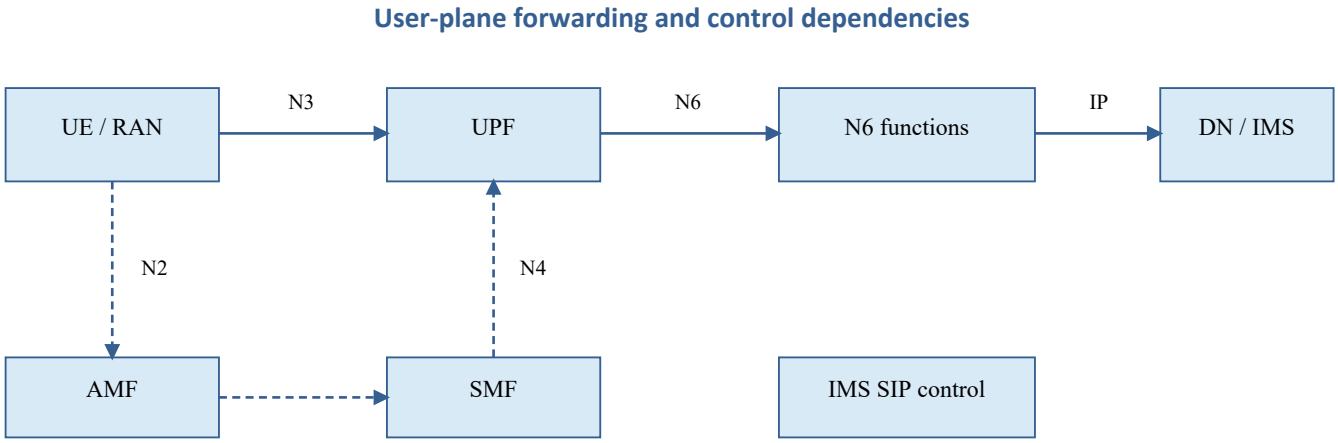
Layer	Components and responsibility
MSP service layer	Provider identity, request ID, service profile, quota and policy. Maps the authenticated principal to a trusted tenant identifier.
Planning layer	R-MRP engine, immutable request, inventory snapshot, rejection reason and deployment plan. Separates hard constraints from preference ranking.
Workload layer	Simulated RAN ingress, UPF representation, DN service endpoints and optional N6 functions. AMF, SMF and IMS have explicit control/service dependencies.
Forwarding and isolation	Tenant addressing, supported NetworkPolicy, security groups and configured UPF/SDN routing. Ordered forwarding is validated separately from allow/deny policy.
AWS operations	EKS/EC2 compute, ECR images, IAM permissions, KMS keys, infrastructure as code, monitoring, logs and lifecycle reconciliation.

The two workload zones in Fig. 1 represent primary and reserved replica chains. EKS control-plane resilience is a separate concern: AWS distributes the managed control plane across three AZs [4]. Worker capacity, application replicas, state replication, failover readiness and recovery procedures remain part of the workload design.

Namespaces organize tenant objects, but they are not sufficient security boundaries on their own. A shared cluster also needs scoped RBAC, quotas, workload restrictions, network policy and careful treatment of privileged networking [5]. Higher-risk tenants may require separate node pools, clusters or accounts; the descriptor must declare the supported isolation class rather than assuming that every tenant can share a node.

The local fixture uses logical AZ-A, AZ-B and AZ-C labels. They are test failure domains, not actual AWS AZ names or a statement about service availability in a chosen Region. All numerical link delays and resource values are supplied planning inputs.

6.1 Service Paths and Telecom Dependencies



Solid: user forwarding. Dashed: selected control links. IMS SIP and RTP paths differ.

Fig. 2. Correct separation of 5G user traffic, control interfaces and IMS signaling.

5G user traffic: UE traffic enters through a simulated RAN, traverses the UPF and reaches the intended data network. Optional N6 service functions, such as firewall or traffic inspection, can appear between UPF and DN. TS 23.501 describes N6-LAN traffic steering to service functions; the forwarding between those functions is deployment-specific [1, clause 5.6.16].

5G control dependencies: the RAN connects to AMF through N2, and SMF controls UPF through N4. Their health and reachability are needed for registration and session setup. They do not become serial hops for every user packet. The full deployment graph therefore contains both forwarding edges and control dependency edges.

IMS signaling: a UE with the required connectivity reaches the IMS access point and uses SIP for registration and session control. A logical signaling route may include P-CSCF, I-CSCF or S-CSCF according to the chosen IMS stack. SIP establishes and modifies sessions [6]; the exact IMS components and interconnections must follow the vendor or open-source implementation.

IMS media: voice or video generally uses an RTP media path negotiated during session establishment [7]. It may traverse a media relay or SBC when the design requires it. It must not be depicted as automatically traversing every SIP proxy. RTP does not reserve bandwidth or guarantee QoS, so the planner and the selected network mechanism must provide the applicable resource and policy controls.

Graph edge type	Validation before activation
User forwarding	Ordered path exists; tenant policy permits all hops; residual link bandwidth and delay bound pass.
Control dependency	Required control endpoint exists; permitted port/protocol and discovery are configured; session-level test passes.
IMS signaling	SIP registration and call setup reach the intended tenant IMS domain; unauthorized domains are denied.
IMS media	Negotiated media endpoints are reachable; authorized media policy and load tests confirm the intended path.

A NetworkPolicy can permit or deny supported connections, but it cannot by itself make traffic visit service functions in order [8]. The deployment adapter must configure forwarding through an explicit mechanism, such as UPF rules and an appropriate routing, SDN or proxy implementation. Policy tests and path-order tests are consequently separate acceptance checks.

6.2 Working Process

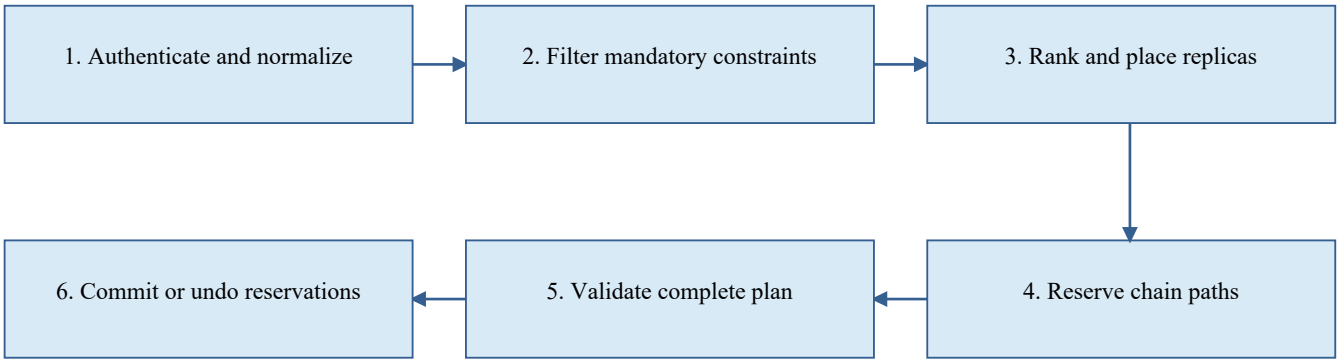


Fig. 3. Planning workflow. A failed candidate returns to the ranked search; a failed request leaves live state unchanged.

1. **Request intake:** read an immutable descriptor, bind the authenticated principal to its tenant, reject unknown fields or invalid units and derive a stable request key.

2. **Policy validation:** verify allowed service roles, tenant quota, isolation class, required zones and the requested replica count before searching.

3. **Resource discovery:** load host residual capacity and network links from one versioned snapshot. Exclude stale, unavailable or incompatible inventory.

4. **Placement search:** rank eligible candidates, place one function at a time and subtract its requirements from tentative host capacity. Enforce replica-zone rules.

5. **Path construction:** connect consecutive service roles in their required order using permitted routes and current tentative link capacity. Add assumed link and processing delays.

6. **Validation:** check the complete request, including each replica and every chain segment. If a branch fails, undo its tentative changes and try the next candidate.

7. **Admission:** return a complete plan and decision trace. A production commit rechecks the inventory version and atomically stores reservations and plan state.

8. **Deployment and activation:** create infrastructure and workloads, apply forwarding and security, then activate only after connectivity, tenant-isolation and service health tests pass.

9. **Monitoring and termination:** reconcile actual state with desired state, apply validated recovery or scaling plans, and release reservations when resources are safely removed.

The conceptual lifecycle is RECEIVED, VALIDATED, PLANNED, RESERVED, PROVISIONING, VERIFYING and ACTIVE. A failure leads to REJECTED before admission or to ROLLING_BACK after provisioning starts. The local prototype covers planning and synthetic reservation, while cloud lifecycle states are proposed implementation behavior.

A retry must not create a second deployment for the same successful request key. In production, persist the request key, plan version, state transitions and provider ownership in a transactional store. Do not derive authorization from a tenant string supplied by an untrusted request alone.

6.3 Algorithm

6.3.1 Problem Model and Input Contract

R-MRP receives a service request S and inventory R. The general service model is a graph whose vertices are required function instances and whose forwarding edges specify the required visit order. The implemented baseline fixes that graph to INGRESS -> UPF -> DATA. INGRESS is an existing endpoint; UPF and DATA are the two placed roles. Each complete chain stays within one zone, and the requested replicas occupy distinct permitted zones. Additional function roles and dependency types are extensions of the architecture, not capabilities claimed for the prototype.

Symbol or field	Meaning and units
tenant / request_id	Authorized provider and stable request identity. Used for ownership and audit records.
cpu / ram / disk	Per-instance required CPU units, memory in GiB and disk in GiB. Prototype values are abstract allocations.
roles	Ordered chain of roles such as INGRESS, UPF and DATA; optional firewall is an additional role.
replicas / allowed_zones	Required number of independent replica chains and the permitted failure domains.
bandwidth	Reserved traffic demand on each traversed directed link, in Mbps in the synthetic fixture.
max_delay	Upper bound on modeled end-to-end chain delay, in ms; not an over-the-air 5G latency target.
hosts / links	Residual host resources, supported roles, tenant compatibility, AZ, directed bandwidth and delay.
search_budget	Maximum attempted placement expansions before returning a budget outcome; bounds runtime.

The prototype requires positive per-role CPU, memory, disk and processing-throughput demands, positive bandwidth and delay limits, and a positive replica count. It supports exactly UPF and DATA demand entries. The trusted fixture supplies host and link inventories; production schema validation and authentication are separate required adapters. Subscriber counts are not converted into compute demand without a sizing model: this study accepts supplied per-function resource requirements.

6.3.2 Mandatory Rules

Rule	Condition for admission
R1 Tenant and quota	Prototype tenant labels match eligible hosts; total new CPU allocations, including replicas, fit the tenant quota. Authentication is an external requirement.
R2 Function compatibility	Host supports the requested role and the required isolation class; unavailable hosts are excluded.
R3 Host capacity	Tentative residual CPU, RAM, disk and processing throughput remain non-negative after every placement.
R4 Tenant boundary	Placed hosts belong to the tenant. Network/link authorization and runtime packet isolation are required deployment controls, outside the prototype.
R5 Failure domains	Replica chains meet the requested distinct-AZ condition; no silent downgrade to one zone.
R6 Path resources	Every consecutive pair has an ordered, permitted path with sufficient cumulative residual bandwidth.
R7 Delay and completion	Sum of supplied link delays meets the bound; all required functions and replicas are present. Processing and queuing delay are not modeled.

Structural tenant, role, AZ and delay filters precede ranking. Dynamic resource and bandwidth checks occur during tentative reservation. A resource with excellent headroom or a low cost proxy cannot override a failed constraint, even if it appears first in the ranked search.

6.3.3 Preference Scoring and Determinism

Preference scores rank structurally eligible complete UPF/DATA chain candidates. Dynamic capacity and bandwidth checks must still pass before a candidate is reserved. Each feature is dimensionless. The reference implementation uses the mean residual CPU fraction of the two hosts after a replica allocation, a normalized zone cost proxy and complete fixed-link chain delay relative to the request limit. The exact coefficients and arithmetic appear in Section 8.2.

Preference	Interpretation	Design caution
Capacity headroom	Prefer hosts with greater remaining capacity after placing the function.	Compute against current tentative state, not the original inventory.
Cost proxy	Prefer the lower supplied relative cost index.	A dimensionless fixture value is not an AWS price or cost estimate.
Delay proxy	Prefer lower complete fixed-link chain delay.	Link-delay sum is a hard check; processing and queuing are excluded.
AZ suitability	Use only zones allowed by request and replica policy.	AZ diversity is mandatory; scoring cannot compensate for its absence.
Stable tie-break	Use lexical host IDs and a fixed function order when scores match.	Avoid unordered iteration and changing wall-clock values in the score.

The same request, inventory, weights and search budget must produce the same plan. The planner records the candidate score and the applicable rejection reason. Stable ordering also makes a changed output explainable: the reviewer can identify which inventory value, policy or configuration changed.

6.3.4 Search and Path Reservation

A ranked greedy decision can prevent admission of a later replica. R-MRP therefore uses depth-first bounded backtracking over complete chain candidates. It reserves UPF, DATA and the chain links on a copied snapshot, then places the next replica in another zone. If a later replica fails, the earlier tentative branch is discarded and another chain candidate is explored. The first complete feasible plan is accepted; global optimality is not claimed.

The prototype computes one deterministic shortest-latency route for INGRESS to UPF and one for UPF to DATA, using lexical path ties. It checks residual bandwidth on those fixed routes; it does not try a longer alternate route if the shortest route is full. Every edge occurrence consumes bandwidth. Opposite directions are separate resources. Route alternatives are a future extension.

Two replica chains receive full-rate reservations in the local model. The standby reservation therefore consumes capacity even if no standby traffic is currently sent. This conservative choice provides planned failover headroom within the fixture, but it does not implement failover or establish application state consistency. Shared prefixes are not deduplicated: independent replicas reserve independent traffic demand.

When the search budget is exhausted, the prototype returns SEARCH_BUDGET rather than a proof that no solution exists. Exhausting its candidates establishes failure only under the fixed-routing, same-zone chain model. A different route, a larger zone set or different placement rules may admit the request.

6.3.5 R-MRP Pseudocode

```
R-MRP(request, inventory, current_state, budget=64)
  validate_positive_demands_and_two_supported_roles(request)
  check_total_new_CPU_against_tenant_quota(request)
  check_number_of_permitted_zones(request)
  C = []
  for (upf, data) in host_pairs(inventory):
    if not tenant_role_and_same_zone_rules(upf, data): continue
    p1 = shortest_latency_route(INGRESS, upf)
    p2 = shortest_latency_route(upf, data)
    if routes_exist_and_total_delay_fits(p1, p2):
      C.append(complete_chain(upf, data, p1, p2))
  attempts = 0
  search(snapshot, selected):
    if len(selected) == request.replicas: return snapshot, selected
    ranked = score_and_sort(C excluding already_selected_zones)
    for chain in ranked:
      if attempts >= budget: return SEARCH_BUDGET
      attempts += 1
      trial = copy(snapshot)
      if not reserve_four_host_resources(trial, chain): continue
      if not reserve_each_directed_edge(trial, chain): continue
      answer = search(trial, selected + chain)
      if answer is complete: return answer
      if answer is SEARCH_BUDGET: return SEARCH_BUDGET
      # Discard trial to roll back this branch
    return NO_FEASIBLE_BRANCH
  result = search(copy(current_state), [])
  if complete: add_tenant_CPU_and_return_plan_snapshot(result)
  else: return_reason_and_unchanged_state(result)
```

The input state is never modified, even after successful planning. The caller explicitly adopts the returned snapshot only on acceptance. Returning a plan is separate from provisioning AWS resources. Production admission additionally needs a version check or lock so concurrent requests cannot both reserve the same residual capacity.

6.3.6 Complexity Analysis

Let H be hosts, V/E the fabric vertices/links, C eligible complete-chain candidates, R the replica count, L the maximum route length and S the number of allocation counters. The two-role implementation inspects at most H squared host pairs and computes two Dijkstra routes per eligible pair. Conventional adjacency-list Dijkstra gives preprocessing of O(H squared (E + V) log V); the prototype also compares and copies path tuples for deterministic ties, adding path-length overhead.

Without a search limit, replica assignment may explore $O(C^R)$ branches. With at most B attempted expansions, search work is O(B(C log C + S + L)) after candidate preprocessing: ranking, state copies and link reservations dominate. Retained search state is O(RS + CL + B), apart from the input topology. The default budget is 64 candidate attempts. These are implementation-oriented bounds, not a polynomial guarantee of globally optimal placement.

6.4 Implementation Approach

The reference implementation is a local Python planning engine. It uses structured fixtures and no cloud credentials. The proposed AWS implementation adds adapters around the planner rather than embedding infrastructure calls inside the candidate-scoring loop. This separation allows a rejected request to leave both the plan state and the cloud environment unchanged.

Module	Implementation responsibility
Request and policy adapter	Validate descriptor schema and units, bind trusted tenant identity, expand requested functions and verify quotas.
Inventory adapter	Read permitted capacity, AZ topology and link estimates; attach version and freshness timestamp; exclude unhealthy resources.
Planner	Apply mandatory rules, deterministic ranking, bounded search and tentative host/link accounting.
Reservation store	Persist plan, request key, ownership, inventory version and reservations atomically; release on safe termination.
Infrastructure adapter	Use CloudFormation/CDK and approved manifests or Helm charts to provision networking and workload resources.
Forwarding adapter	Configure supported UPF rules and the selected N6 forwarding mechanism; apply explicit tenant allow/deny policies.
Validation adapter	Run health, chain-order, reachability, tenant-denial and optional SIP/media checks before activating a service.
Operations adapter	Collect metrics and events, reconcile desired state, and create a new validated plan for scaling or recovery.

6.4.1 AWS and Kubernetes Mapping

EKS can host compatible containerized functions, while EC2 can host VM-based functions or specialized networking workloads. ECR stores approved images. VPC subnets, routing and security groups provide infrastructure-level connectivity controls. IAM scopes API actions, and KMS supports encryption where the selected service integrates with it. CloudWatch and workload telemetry provide observability; domain-level 5G and IMS probes must be added explicitly.

Use tenant labels and restricted service accounts, ResourceQuota, appropriate Pod Security controls and supported NetworkPolicy. Protect policy and tenant labels from unauthorized modification. Topology spread constraints or required pod anti-affinity can enforce replica separation; Kubernetes documents DoNotSchedule for mandatory spread behavior [9]. Deployment must remain pending or fail admission if the required capacity is unavailable.

6.4.2 Rollback and Concurrent Admission

Planning rollback discards tentative allocations. Deployment rollback must additionally undo cloud and Kubernetes objects created by the request, in reverse dependency order, while preserving resources owned by other tenants. Durable operation identifiers and ownership tags are required. If the inventory version changes before commit, re-read the state and replan; never apply a stale reservation merely because it was previously feasible.

7. ENVIRONMENTAL SETUP

7.1 Hardware Requirements

Hardware	Recommended academic setup	Purpose
Processor	Modern 4-core or better CPU	Local development, containers and planning tests.
RAM	16 GB minimum; 32 GB preferred for emulation	Concurrent development tools and optional network workloads.
Storage	At least 100 GB free SSD for full laboratory tools	Images, emulators, traces, logs and artifacts.
GPU	Not required	R-MRP is rule-based and does not train an ML model.
Network	Stable broadband for cloud phase	AWS API access, image transfer and remote monitoring.
Optional device	UE/gNB emulator or supported test device	Later end-to-end telecom validation.

The planner itself is lightweight and does not require the full emulation setup. The larger requirements support a future laboratory that runs containers and network functions. A local planner result can be reproduced without an AWS account or physical 5G radio hardware.

7.2 Software Requirements

Software or service	Role in the study	Status
Python 3.11 or later	Standard-library reference planner and test driver.	Implemented locally.
JSON	Portable synthetic inventory, request and result records.	Used for reproducibility.
Windows / Linux	Development operating system.	Local execution environment.
AWS CLI and IaC tools	Infrastructure provisioning through approved APIs.	Proposed deployment phase.
Docker and compatible Kubernetes / EKS	Container packaging and workload orchestration.	Proposed laboratory/cloud phase.
VPC, EC2, ECR, IAM, KMS	Networking, compute, image, identity and encryption support.	Architecture components.
CloudWatch and workload probes	Infrastructure metrics plus telecom/service health.	Proposed validation phase.
5GC, RAN and IMS software	Protocol-level integration and load testing.	Must be selected and validated separately.

Versions and instance types should be selected for the actual Region, workload software and required network capabilities. This report does not fix a current AWS price or claim that every network function can run on an arbitrary general-purpose node. Vendor support, packet processing requirements and acceleration constraints must be reflected in host eligibility rules.

7.3 Reproducibility and Data Handling

The supporting package contains the planner, tests, fixture and generated result records. Inputs are artificial tenant IDs and resource values. Test execution is deterministic except for optional timing observations; timing is not used to rank candidates. Keep credentials, subscriber identifiers and production network inventories out of shared academic fixtures.

8. TARGET

The targets below describe desired behavior for a future deployment. They are acceptance criteria, not claimed AWS or 5G measurements. The executed local tests in Section 9 validate a narrower planning model using synthetic resources and assumed network delays.

Output	Target behavior	Evaluation method
Admission safety	Every accepted plan satisfies all configured mandatory constraints.	Independent invariant checks on every plan.
Invalid request handling	Constructed identity, quota, capacity and tenant violations are rejected.	Negative fixtures with expected reason.
Repeatability	Equivalent request and state produce the same normalized plan.	Repeat and compare serialized plans.
Resource accounting	Host and link residuals remain non-negative, including replicas.	Recompute usage from selected assignments and paths.
AZ diversity	Required replica chains occupy distinct permitted zones.	Inspect placements and remove one zone in a controlled test.
Service-chain correctness	Packets follow the intended ordered service functions.	Later packet tracing and service-function hit counters.
Operational isolation	Unauthorized cross-tenant traffic and API actions are denied.	Later network, IAM and RBAC denial tests.
Recovery	Configured surviving replica can serve new traffic after a controlled fault.	Later failover test with observed interruption and state checks.

Evaluation Protocol

For the algorithm phase, run each test from a fresh inventory unless the test intentionally checks sequential admissions. Record the complete descriptor, inventory version, policy settings, expected outcome and actual outcome. A rejection test succeeds when the planner rejects the invalid request and leaves the live state unchanged; it is not counted as a failed deployment.

For the cloud phase, separate planning time, API provisioning time, workload readiness time and service activation time. Report the number of requests, admitted requests, rejected invalid requests, retries and search-limit outcomes. Use medians and tail percentiles only after enough repeated observations have been collected. Do not report a single synthetic fixture as a population-level success percentage.

Metric	Definition and interpretation
Plan validity	Accepted plans passing all independent checks / accepted plans tested.
Correct negative handling	Invalid fixtures rejected with unchanged state / invalid fixtures tested.
Planning latency	Wall-clock duration of the planner only; measured separately from cloud operations.
Search effort	Attempted candidate expansions and path alternatives; helps explain difficult requests.
Utilization	Reserved CPU, RAM and link bandwidth divided by the corresponding modeled capacity.
Recovery interruption	Observed time from injected fault until a validated service is usable; not measured here.

8.1 Sample Input and Output

The reference fixture uses three logical zones and role-specific hosts. The request asks for two complete chains in distinct zones. Each chain reserves capacity independently. INGRESS is a fixed synthetic endpoint, UPF is a placement role rather than a running packet forwarder, and DATA is a tenant service endpoint. The executable fixture uses tenant alpha; this is an artificial provider identifier, unrelated to personal identity.

Request field	Worked-example value
Tenant	alpha
Service	Ordered INGRESS -> UPF -> DATA baseline; variable-length chains are a future extension.
Replicas	Two complete chains in distinct AZs.
Traffic demand	20 Mbit/s per replica; full-rate reservations for both copies.
Capacity	Per role: 2 vCPU, 4 GiB RAM, 10 GiB disk, 100 Mbit/s processing allocation.
Latency bound	12 ms maximum fixed-link delay; processing and queuing excluded.
Security	UPF and DATA hosts must match tenant alpha. Runtime access control is not simulated.
Output	Placement, route list, returned allocation snapshot, score trace and admission status.

Executable Request Fields

```
{
  "id": "MSP-001", "tenant": "alpha",
  "replicas": 2,
  "allowed_zones": ["A", "B", "C"],
  "bandwidth": 20, "latency_limit": 12,
  "demands": {
    "UPF":  {"cpu":2, "memory":4, "disk":10, "throughput":100},
    "DATA": {"cpu":2, "memory":4, "disk":10, "throughput":100}
  }
}
```

These are the JSON-serialized fields of the prototype Request data class. A deployment descriptor additionally needs approved image references, protocol endpoints, persistent-state requirements and function-specific health probes. Those operational fields must not be guessed from a placement role name.

Interpreting the Output

An admitted plan identifies the host assigned to each function in each replica, the sequence of directed links for each chain segment, the remaining host and link capacity, and a reason for each placement. The response does not claim DEPLOYED or ACTIVE merely because the planner found a solution. A production orchestrator advances those states only after provisioning and service validation.

The numerical worked example and test matrix that follow are populated from the executed reference implementation. All values describe the local synthetic model, not AWS capacity guarantees or measured telecom traffic.

8.2 Numerical Worked Example

All numbers in this worked example are synthetic inputs to the local reference model. They make the decision trace reproducible; they are not measurements of AWS or 5G equipment. Six tenant-alpha hosts are distributed over three zones, with one UPF host and one DATA host per zone.

Zone	Hosts	Capacity of each host	Cost proxy
A	UA / DA	8 vCPU; 32 GiB RAM; 200 GiB disk; 1,000 Mbit/s processing	0.6
B	UB / DB	8 vCPU; 32 GiB RAM; 200 GiB disk; 1,000 Mbit/s processing	0.3
C	UC / DC	8 vCPU; 32 GiB RAM; 200 GiB disk; 1,000 Mbit/s processing	0.1

Directed link group	Assumed delay	Capacity
INGRESS -> S shared trunk	1 ms	80 Mbit/s
S -> RA / RB / RC	1 / 3 / 6 ms respectively	100 Mbit/s each
Router -> local UPF or DATA host	1 ms	100 Mbit/s each
Local host -> router	1 ms	100 Mbit/s each

A chain follows INGRESS -> S -> local router -> UPF -> local router -> DATA. The resulting fixed-link delays are 5 ms in A, 7 ms in B and 10 ms in C. UPF and DATA processing and queuing delay are excluded. Each chain reserves 20 Mbit/s on every edge occurrence. The 100 Mbit/s per-host processing allocation is a separate abstract host constraint, not the same counter as network traffic.

8.2.1 Initial Candidate Scores

The reference score is 50 times h plus 30 times (1 - L/Lmax) plus 20 times (1 - c). Here h is the mean post-placement residual CPU fraction on the two hosts, L is chain link delay, Lmax is the 12 ms request limit, and c is the supplied zone cost proxy. The coefficients sum to 100. All three chains satisfy the initial structural rules.

Zone	CPU term	Delay term	Cost term	Total score
A	50 x 0.75 = 37.5	30 x (1 - 5/12) = 17.5	20 x 0.4 = 8.0	63.0
B	50 x 0.75 = 37.5	30 x (1 - 7/12) = 12.5	20 x 0.7 = 14.0	64.0
C	50 x 0.75 = 37.5	30 x (1 - 10/12) = 5.0	20 x 0.9 = 18.0	60.5

For every initially unused host, post-placement CPU headroom is (8 - 2)/8 = 0.75. B ranks first despite a longer path than A because its lower cost proxy offsets the delay preference. A ranks next. Once B is selected, a second B chain is excluded by the distinct-AZ rule. The first request therefore selects B and A after two candidate attempts.

The cost input is a preference coefficient, not currency. Changing it changes a policy choice; it does not establish a monetary saving. A latency limit below a candidate path delay removes that candidate before scoring, regardless of its cost or headroom.

8.2.2 Placement and Reservation Output

Replica	UPF / DATA	Zone	Link delay	Score
1	UB / DB	B	7 ms	64.0
2	UA / DA	A	5 ms	63.0


```
{
  "accepted": true,
  "reason": "ACCEPTED",
  "attempts": 2,
  "placements": [
    {"zone":"B", "upf":"UB", "data":"DB", "latency":7},
    {"zone":"A", "upf":"UA", "data":"DA", "latency":5}
  ],
  "tenant_cpu_reserved": 8,
  "shared_trunk_reserved_mbps": 40
}
```


The compact response above is an excerpt of the complete metrics record. Four placed role instances reserve 8 vCPU, 16 GiB RAM, 40 GiB disk and 400 Mbit/s of aggregate host-processing allocation. Two independent replicas reserve 2 x 20 = 40 Mbit/s on the shared ingress trunk. The original input state remains unchanged; the caller can adopt the returned snapshot.

8.2.3 Sequential Admissions and Bottleneck Analysis

Request / policy	Result	Zones	Tenant CPU	Trunk use
First, CPU quota 16	Accepted	B, A	8 / 16	40 / 80 Mbit/s
Second, same policy	Accepted	C, B	16 / 16	80 / 80 Mbit/s
Third, quota 16	QUOTA	None	16 / 16	80 / 80 Mbit/s
Third, quota raised to 100	CAPACITY_OR_DIVERSITY; trace identifies trunk bandwidth	None	16 / 100	80 / 80 Mbit/s

After the first request, A and B each have 2 vCPU used on both role hosts. For a second request their post-placement headroom falls to 0.5, whereas C retains 0.75. Scores become A = 50.5, B = 51.5 and C = 60.5. The second request selects C and B. Host CPU usage becomes UA/DA = 2 each, UB/DB = 4 each and UC/DC = 2 each.

Only 16 of 48 host vCPUs are reserved after two requests, but the shared trunk is fully reserved at 80 Mbit/s. This is the key placement lesson: spare CPU does not imply that another service chain is admissible. Raising the CPU quota exposes the independent network bottleneck rather than resolving it. Both rejection paths preserve the previously admitted reservations.

The shared INGRESS and S nodes also form a common modeled failure dependency. Distinct workload zones do not provide complete end-to-end fault tolerance when all chains share an unprotected ingress path. A production design needs redundant access paths and verified failover in addition to application placement.

9. RESULTS AND DISCUSSION

The reference implementation was executed locally using its supplied synthetic fixture. The test suite validates admission logic and state accounting. It does not execute packets, deploy Kubernetes workloads, provision AWS resources or run a 5G/IMS stack. The following matrix records the behavioral checks and their observed outcomes.

Test	Expected behavior	Observed
Valid request and arithmetic	Select B/A, scores 64/63, delays 7/5 ms, CPU 8 and trunk 40.	PASS
Tenant CPU quota	Quota 7 cannot admit an 8-vCPU request; no state change.	PASS
Four host resource dimensions	Insufficient CPU, memory, disk or processing throughput independently rejects.	PASS
Tenant host compatibility	Tenant beta cannot use alpha-owned hosts.	PASS
Impossible latency	4 ms bound rejects every available chain.	PASS
AZ diversity	One permitted zone rejects two replicas; A/C permits distinct replicas.	PASS
Determinism	Identical request and state yield identical plan and trace.	PASS
Cumulative host CPU	Second admission preserves the first reservations and selects C/B.	PASS
Shared trunk bandwidth	Both replicas and existing requests consume the same trunk counter.	PASS
Partial failure rollback	30 Mbit/s trunk cannot fit two 20 Mbit/s replicas; undo partial branch.	PASS
Search budget	One candidate attempt returns SEARCH_BUDGET with unchanged state.	PASS
Role eligibility	Removing UPF capability from UB excludes its B chain.	PASS
Malformed numeric request	Reject non-finite or invalid numeric request values before planning.	PASS

The suite contains 13 tests. The resource-dimension test includes separate CPU, memory, disk and throughput subcases, and the malformed-input test includes multiple invalid-value subcases. These are software test outcomes, not a measured deployment-success rate. No confidence interval or production acceptance percentage can be inferred from this small deterministic suite.

Interpretation of the Evidence

The accepted plans match the manual score arithmetic and account for full replica capacity. Sequential admission changes placement because residual headroom changes. The quota and bandwidth examples show that rejection reasons depend on which independent constraint fails first. The rollback test is particularly useful: the first replica can be tentatively placed, but failure to place the second must reject the whole request without leaking reservations.

A finite search budget limits effort but can reject a feasible request when set too low. In the budget test, one attempt cannot complete a two-replica plan. This is intentionally reported as incomplete search, not a definitive infeasibility result. A caller could retry with a larger budget under an explicit policy.

Comparison and Sensitivity Analysis

Criterion	CPU-only or manual decision	R-MRP design / demonstrated scope
Host selection	May check a host independently without later allocations.	Tentative counters include earlier functions and requests.
Chain connectivity	Requires a separate manual route assessment.	Fixed shortest routes are computed and reserved in the local model.
Bandwidth	Spare compute may be treated as sufficient.	Shared trunk is checked across replicas and admitted requests.
Failure domains	Replica count can hide same-zone placement.	Distinct permitted AZs are mandatory in the model.
Failed admission	Manual partial changes need explicit cleanup.	Copied state prevents failed planning branches from leaking allocations.
Explainability	Decisions may depend on unrecorded judgment.	Stable scores, placements, attempts and rejection traces are recorded.
Runtime service correctness	Requires service and network measurements.	Still requires external deployment and end-to-end validation.

This comparison describes design characteristics, not an executed timing or cost benchmark against a manual team. The study does not claim a measured provisioning-time reduction or percentage cost saving.

Sensitivity to Policy and Capacity

Delay constraint: with the supplied link inputs, A requires 5 ms, B 7 ms and C 10 ms. A bound below 7 ms leaves at most A and cannot satisfy two distinct-zone replicas. This follows analytically from the fixture and explains why latency is a hard admission condition rather than a preference alone.

Preference weights: initial B exceeds A by only one score point. Increasing the relative weight of delay or changing the cost proxy can make A rank first without changing feasibility. Such changes must be stored in the policy version so decisions remain reproducible.

Trunk capacity: each request needs 40 Mbit/s for two full-rate chains. An 80 Mbit/s trunk can accommodate two such requests by this counter, while a 30 Mbit/s trunk cannot accommodate even one. These are model-derived capacity relationships; actual traffic distribution and failover policy may require a different reservation rule.

Replica policy: complete chains are restricted to one zone each in the prototype. Relaxing that locality restriction might increase admission options, but would add cross-zone links and could change both delay and failure exposure. Such a change requires new tests rather than a simple score adjustment.

The baseline is best suited to small, policy-controlled admission problems where traceability matters. For larger topologies, cache reusable routes, limit candidates and use an explicit alternate-path policy. More advanced optimization should be compared against the same constraints and workload set; it should not be judged only by a higher admission count that comes from weaker rules.

9.1 Advantages

• **Explainable admission:** every decision can be related to a tenant rule, resource counter, route, delay bound or preference score.

• **Repeatability:** fixed inputs and deterministic tie-breaking produce the same plan and trace in the local tests.

• **Complete-request accounting:** CPU, memory, disk, processing capacity and directed links are reserved for all replica chains.

• **Controlled failure behavior:** failed branches and rejected requests preserve the previous allocation state.

• **Explicit failure domains:** replica diversity is mandatory rather than an optional score preference.

• **Practical implementation path:** the planner can be wrapped with cloud, policy, forwarding and validation adapters without requiring an ML model.

9.2 Limitations

• **Restricted executable scope:** only a fixed INGRESS endpoint and placed UPF/DATA roles are modeled. General service graphs, separate firewall resources and IMS function decomposition remain extensions.

• **Fixed routing:** only one shortest-latency route per segment is used. An alternative route with spare bandwidth is not explored.

• **Synthetic delay:** link-delay sums exclude radio, processing, queuing, packet loss and protocol overhead. They are not end-to-end 5G measurements.

• **Limited isolation model:** host tenant labels constrain placement. Authentication, tenant-aware link ACLs, network enforcement, storage isolation and cryptographic controls are external requirements.

• **Availability is incomplete:** the fixture shares ingress infrastructure and does not simulate state replication, traffic failover, AZ failure or multi-region recovery.

• **No global optimality guarantee:** first-feasible ranked search may use more resources or admit fewer future requests than an optimization-based strategy.

• **Prototype interfaces:** trusted host/link inventories and a single-process caller are assumed. Durable idempotency, concurrency control, inventory reconciliation and comprehensive production schema validation are not implemented.

• **Small evaluation set:** passing deterministic software tests does not prove real-world isolation, scalability, cloud performance, service availability or telecom interoperability.

These limitations define the boundary of the evidence. The contribution is a reproducible, constrained planning procedure with a clear implementation path, rather than a claim that a production telecom platform has already been deployed.

10. CONCLUSION

This case study extended the 5G MSP strategy on AWS by developing its rule-based resource-placement and service-chaining objective into a concrete algorithm and executable reference model. The design turns a tenant request into an explicit plan through host eligibility, resource and quota checks, distinct-zone replicas, ordered paths, bandwidth reservations and deterministic preference ranking.

The local worked example selected B and A for the first request and C and B for the second. After two requests, only one-third of host CPU was reserved, while the shared ingress trunk was fully reserved. A third request was consequently rejected by quota or bandwidth, depending on the configured quota. This demonstrates why placement must account for network resources and existing reservations as well as compute capacity.

The executed tests support the implemented admission, arithmetic, deterministic behavior and rollback properties. They do not establish physical 5G performance, IMS interoperability or AWS availability. The architecture therefore keeps user forwarding, control dependencies, IMS signaling, infrastructure provisioning and runtime validation separate. An operational deployment must implement and test each of those responsibilities.

R-MRP provides a suitable academic baseline because its decisions are understandable, its assumptions are explicit and its outputs can be reproduced. A later laboratory can retain the same descriptor and constraint model while replacing synthetic inventory with measured infrastructure data and adding protocol-level validation.

11. FUTURE ENHANCEMENT

1. **General service graphs:** support variable-length chains with function-specific resource demands, including N6 firewall or inspection services and explicitly modeled IMS dependencies.

2. **Alternate path selection:** consider a bounded set of routes with residual bandwidth, permitted tenant domains and measured delay.

3. **Edge-aware placement:** add supported edge infrastructure and distinguish radio, transport, processing and application delay budgets.

4. **Production state management:** add persistent request keys, atomic compare-and-swap admission, ownership records and lifecycle reconciliation.

5. **Cloud and telecom integration:** connect approved infrastructure templates, network functions, forwarding adapters and SIP/media validation in a controlled laboratory.

6. **Recovery and state:** model redundant ingress, stateful session recovery, traffic steering during failure and safely released reservations.

7. **Comparative evaluation:** compare the baseline with first-fit and optimization methods under identical policy constraints and reproducible load traces.

8. **Observed operations:** add telemetry-driven capacity estimates and controlled scaling, then assess forecast-based planning only after a validated baseline exists.

12. REFERENCES

[1] ETSI / 3GPP. **TS 123 501 V18.5.0 System architecture for the 5G System 5GS.** May 2024. Clauses 4.2.7 and 5.6.16.
https://www.etsi.org/deliver/etsi_ts/123500_123599/123501/18.05.00_60/ts_123501v180500p.pdf

[2] IETF. **RFC 7665 Service Function Chaining Architecture.** October 2015.
https://www.rfc-editor.org/rfc/rfc7665

[3] Amazon Web Services. **5G core deployment on AWS.** AWS Well-Architected Telco Lens.
https://docs.aws.amazon.com/wellarchitected/latest/telco-lens/g-core-deployment-on-aws.html

[4] Amazon Web Services. **Amazon EKS architecture.** Amazon EKS User Guide.
https://docs.aws.amazon.com/eks/latest/userguide/eks-architecture.html

[5] Kubernetes. **Multi-tenancy.** Kubernetes documentation.
https://kubernetes.io/docs/concepts/security/multi-tenancy/

[6] IETF. **RFC 3261 SIP Session Initiation Protocol.** June 2002.
https://www.rfc-editor.org/rfc/rfc3261

[7] IETF. **RFC 3550 RTP A Transport Protocol for Real-Time Applications.** July 2003.
https://www.rfc-editor.org/rfc/rfc3550

[8] Kubernetes. **Network Policies.** Kubernetes documentation.
https://kubernetes.io/docs/concepts/services-networking/network-policies/

[9] Kubernetes. **Pod Topology Spread Constraints.** Kubernetes documentation.
https://kubernetes.io/docs/concepts/scheduling-eviction/topology-spread-constraints/

[10] Amazon Web Services. **Modernizing IMS networks on AWS.** Umair Rehmat and Sudhir Shet, 20 May 2022. Background reference.
https://aws.amazon.com/blogs/industries/modernizing-ims-networks-on-aws/

Official documentation was consulted for architectural definitions and implementation responsibilities. The placement rules, synthetic fixture, score weights and local results are the case study design and reference-model evidence. The AWS IMS reference [10] supplies additional context for modernization and deployment of IMS workloads.

Appendix A - Key Assumptions

• The original cover, author identity, guide identity and institutional text are retained. The extension develops the original third objective while preserving the main report section sequence.

• RAN access is represented conceptually and by a fixed synthetic ingress endpoint in the code. Physical radio units and over-the-air measurements are outside the baseline.

• One logical infrastructure domain contains three test AZ labels. These labels do not identify an actual AWS Region or AZ.

• Tenant alpha is an artificial provider ID. Tenant ownership is modeled on hosts; real authorization and packet isolation require separate enforcement.

• The prototype uses exactly UPF and DATA placement roles, with both roles in the same AZ for each replica. Two replica chains occupy distinct permitted zones.

• Hosts and directed links have fixed supplied capacities. Delay is the sum of fixed link inputs, with processing and queuing excluded.

• Each replica reserves full traffic demand on every traversed edge occurrence. Shared trunk traffic is counted separately for each replica.

• Preference cost values are dimensionless inputs. No current AWS bill, instance quote or monetary saving is estimated.

• The caller adopts an accepted returned snapshot. Production transaction locking, idempotency and cloud provisioning remain integration work.

• Results are executed local planning tests. Operational deployment targets remain unmeasured.

Appendix B - Reproducing the Reference Model

The supporting package contains planner.py, test_planner.py, metrics.json and TECHNICAL_NOTE.md. A standard Python environment is sufficient; the planner has no third-party dependency or cloud API call. Run the following commands from the package directory:

```
python -m unittest -v test_planner.py
python planner.py
```

The test command reports the behavioral checks. The second command regenerates metrics.json with the exact request, hosts, links, selected chains, returned snapshots and rejection traces. Check that the first zones are B/A, the second zones are C/B, and the cumulative tenant CPU and shared trunk are 16 vCPU and 80 Mbit/s after two accepted requests.

Before a cloud demonstration, add infrastructure and service-function validation, tenant negative tests, forwarding-order checks, workload readiness, stateful failover and measured latency under controlled load. Keep their results separate from the local planning evidence so every conclusion remains traceable to the test that supports it.